Ras Al Khaimah International Airport

Complete IIS Hosting Guide - Website + Flight API + AI Chatbot

This guide explains how to host the entire RAK Airport website on Microsoft IIS. There are three parts: the website itself (static files), the live flight data API (FIDS), and the AI chatbot (Rakia). IIS serves the website and reverse-proxies the two backends, so the browser always stays on one origin - no CORS configuration is needed.

The three components

Component	What it is	Runs on	URL on the site
Website (frontend)	React static files (the dist folder)	IIS	/ (all pages)
Flight API (FIDS)	Your separate AMS flight app	localhost:3005	/getFlights
AI Chatbot (Rakia)	Node server (server/rakia-server.mjs)	localhost:3001	/chat

How requests flow

```
              +--> /            static website (React)
Browser > IIS +--> /getFlights  -> localhost:3005 (FIDS)  -> flight JSON
              +--> /chat        -> localhost:3001 (Rakia) -> AI answer
```

All three are served from the same origin, so there are no cross-origin (CORS) problems, and it works the same over HTTP or HTTPS. The web.config that ships with the build already contains both proxy rules.

Step 0 - Build the site

In the project folder, run:

```
npm install       (first time only)
npm run build
```

This produces the **dist** folder - your deployable website. It is already configured to call /getFlights and /chat on its own origin. Deploy the contents of dist (it includes web.config with the proxy rules).

Step 1 - Prepare IIS (once)

- Install **IIS** (Windows Features > Internet Information Services).
- Install **URL Rewrite**: https://www.iis.net/downloads/microsoft/url-rewrite
- Install **Application Request Routing (ARR)**: https://www.iis.net/downloads/microsoft/application-request-routing
- Enable proxy: IIS Manager > click the **server node** > **Application Request Routing Cache** > **Server Proxy Settings** > tick **Enable proxy** > **Apply**. (Without this, /getFlights and /chat return 502.)

Step 2 - Host the website (frontend)

1. Extract the build into a folder, e.g. **C:\inetpub\rakairport** (index.html, web.config, assets\, images).
2. IIS Manager > right-click **Sites** > **Add Website**. Name: rakairport; Physical path: C:\inetpub\rakairport; Binding: port 80 and your host name.
3. **Application Pools** > the site's pool > **Basic Settings** > **.NET CLR version = No Managed Code**.

4. Give **IIS_IUSRS** Read & execute on the folder.

Step 3 - Run the Flight API (FIDS)

The flight boards call /getFlights, which is served by your separate **FIDS API** app (part of the AMS system) - not by this website.

- Start the FIDS API so it listens on **localhost:3005**.
- Run it as a **Windows service** (NSSM or pm2) so it restarts automatically.
- The web.config proxy already forwards /getFlights to it. If it uses another port, change the target URL in the "FIDS Flights proxy" rule.

Step 4 - Run the AI Chatbot server (Rakia)

Copy the **server** folder (rakia-server.mjs + site-knowledge.json) to the server, then run it on **localhost:3001** with your Groq API key. Run it as a Windows service so it stays up - example with NSSM:

```
nssm install RakiaAI "C:\Program Files\nodejs\node.exe" "C:\rakia-ai\rakia-server.mjs"
nssm set RakiaAI AppEnvironmentExtra GROQ_API_KEY=YOUR_GROQ_KEY PORT=3001
nssm start RakiaAI
```

The web.config proxy already forwards /chat to it. The Groq key stays on the server (in the service environment), never in the website. If you skip this, the chatbot still works using on-site keyword search - just not conversationally.

Step 5 - HTTPS (recommended)

Add an SSL binding (port 443) with a certificate. On IIS you can use **win-acme** (free Let's Encrypt automation) or import your own certificate. The internal proxy hops to localhost:3005 and localhost:3001 can stay HTTP (server-to-server), with no mixed-content issue because the browser only talks to your HTTPS site.

Step 6 - Test everything

- **Website:** open the site, browse to /parking and press F5 - it should load, not 404.
- **Flights:** on the server, open http://localhost:3005/getFlights?type=ARR - it should return flight JSON; the Arrival/Departure boards then populate and refresh every 60 seconds.
- **AI chat:** on the server, open http://localhost:3001/health - it should show {ok:true, key_found:true}; then open the chatbot and ask a question for a live answer.

Troubleshooting

- **500.19** - URL Rewrite module not installed.
- **502 on /getFlights or /chat** - ARR proxy not enabled, or that backend service is not running on its port.
- **404 on refresh of a deep page** - web.config missing, or URL Rewrite not installed.
- **Chatbot not conversational** - the Rakia AI service (3001) is not running, or the Groq key is missing; it falls back to keyword search.
- **Image not updating after replace** - browser/CDN cache; rename the file (add -v2).